

# MF703: Bitcoin Price Prediction Research Codebase

---

**Author:** BTC SATOSHI LAB

**Last Updated:** 2025-12-18

---

## Overview

This repository contains the complete codebase for the bitcoin prediction research project focused on **eda**, **time-series forecasting**, **factor engineering(icir)**, **modeling** and **systematic trading strategy backtesting** using high-frequency (1-minute) financial data.

The project emphasizes:

- **Modularity:** clear separation between data processing, modeling, and evaluation
- **Reproducibility:** deterministic pipelines and explicit data/model organization
- **Comparability:** unified backtesting logic across different model classes

Models implemented range from traditional econometric approaches (ARIMA, OLS/Ridge/Lasso) to machine learning and deep learning methods (XGBoost, CNN-LSTM).

---

## Project Structure

```
codebase/
├── data/                                # Raw and intermediate datasets (1-min
frequency)
│   ├── btcusd_1-min_data.csv
│   ├── ETH_USD_1min_2020_2025.csv
│   ├── SPY_1min_2020_2025.csv
│   └── ...
├── data_processing/                    # Shared data loading & preprocessing
utilities
│   └── data_loader.py
├── eda/                                # Exploratory data analysis
│   └── eda_main.ipynb
├── feature_engineering_icir/          # Factor construction and IC / ICIR
evaluation
│   ├── feature_engineering.py
│   ├── new_factors_ic_icir_test.py
│   └── New_factors_IC_ICIR_Test.ipynb
├── models/                            # Model-specific implementations
│   ├── arima/
│   │   └── arima_main.ipynb
```

```

├── data_loader.py
├── feature_engineering.py
├── linear_lasso_ridge/
│   ├── data_loader.py
│   └── ols_lasso_ridge_main.ipynb
├── xgboost/
│   ├── data_loader.py
│   ├── backtesting.py
│   └── xgboost_main.ipynb
├── cnn_lstm/
│   └── cnn_lstm.py           # the hybrid cnn-lstm structure, helper
functions
├── modeling.py
├── backtesting.py
├── cnn_lstm_main.ipynb
├── lstm_best_model.pth
├── common_modeling.py       # Shared modeling utilities (e.g. metrixs,
plot_valid_loss..)
├── backtesting/            # Unified backtesting framework and
outputs
│   ├── backtesting.py      # cotain functions like get_signals,
get_backtest_metrics
│   ├── backtesting_main.ipynb
│   └── backtest_outputs.csv
├── results/                # Clean notebooks for result presentation
│   ├── arima_main.ipynb
│   ├── backtesting_main.ipynb
│   ├── cnn_lstm_main.ipynb
│   ├── eda_main.ipynb
│   ├── ols_lasso_ridge.ipynb
│   └── xgboost_main.ipynb

```

## Modeling Pipeline

A typical workflow follows:

### 1. Data Loading & Cleaning

Handled via [data\\_processing/data\\_loader.py](#) or model-specific loaders

### 2. Feature Engineering

Technical indicators, statistical transforms, and factor IC/ICIR evaluation

### 3. Model Training

- Linear models: OLS / Ridge / Lasso

- Time-series models: ARIMA
- ML models: XGBoost
- DL models: CNN-LSTM

#### 4. Signal Construction

Continuous predictions transformed into discrete trading positions

#### 5. Backtesting & Evaluation

Unified backtesting logic to ensure fair comparison across models

---

## How to Run

Typical workflows are executed via Jupyter notebooks located in the `models/` and `results/` directories. For example:

- ols\_lasso\_ridge: `models/linear_lasso_ridge/ols_lasso_ridge_main.ipynb`
  - arima: `models/arima/arima_main.ipynb`
  - XGBoost: `models/xgboost/xgboost_main.ipynb`
  - CNN-LSTM: `models/cnn_lstm/cnn_lstm_main.ipynb`
  - Backtesting summary: `results/backtesting_main.ipynb`
- 

## Notes

- All models operate on minute-level financial data.
  - Model-specific backtesting logic is implemented where necessary, while a unified framework is provided in `backtesting/`.
  - The codebase is organized to facilitate reproducibility, extensibility, and fair comparison across modeling approaches.
- 

## Dependencies

Key dependencies include (non-exhaustive):

- Python 3.11
  - NumPy, Pandas
  - scikit-learn
  - statsmodels
  - PyTorch
  - XGBoost
- 

## Author Notes

This repository is structured for academic research and quantitative strategy experimentation, with an emphasis on clarity, modularity, and reproducibility.